

Bulk Operations in Entity Framework Core using EFCore.BulkExtensions

In this article, I will discuss **Bulk Operations in Entity Framework Core using EFCore.BulkExtensions** Extension with Examples. Please read our previous article, which discussed How to perform [Bulk Operations in EF Core using Z.EntityFramework.Extensions.EFCore](#) Extension with Examples.

Bulk Operations in Entity Framework Core using EFCore.BulkExtensions

EFCore.BulkExtensions is an open-source library that extends Entity Framework Core (EF Core) by providing high-performance bulk operations. When dealing with large amounts of data, standard EF Core operations can be inefficient due to multiple database round trips and extensive change tracking. EFCore.BulkExtensions addresses these issues and helps improve the performance of database interactions, especially when dealing with large datasets, by reducing the number of database round trips and optimizing SQL commands.

This library provides several methods for inserting, updating, deleting, or merging data in bulk, making it an ideal choice for applications that require frequent and efficient handling of large volumes of data. EFCore.BulkExtensions supports multiple relational databases, such as SQL Server, PostgreSQL, MySQL, and SQLite.

You can find more information and contribute to the library via its GitHub repository: GitHub Link: <https://github.com/borisdj/EFCore.BulkExtensions>

Bulk Operations Methods Provided by EFCore.BulkExtensions Package:

EFCore.BulkExtensions provides several methods to enhance the performance of bulk database operations. These methods are:

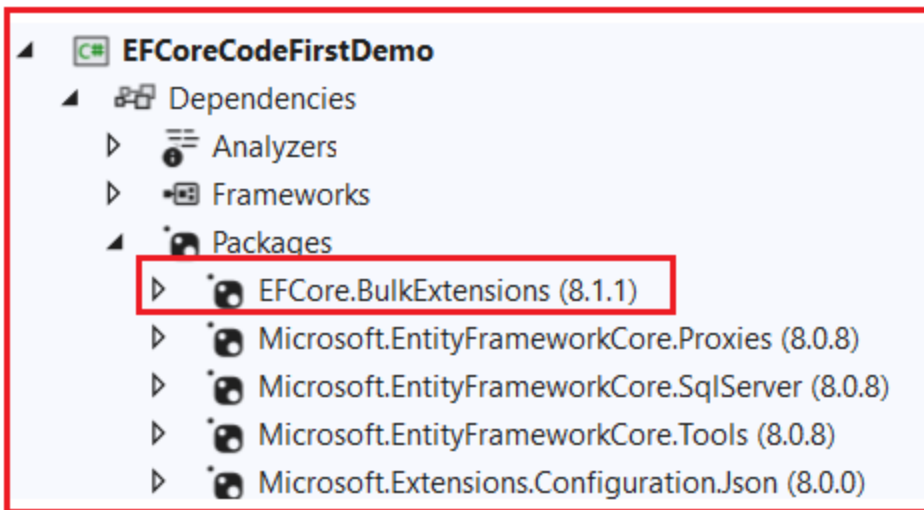
- **BulkInsert / BulkInsertAsync:** Inserts multiple records into the database in a single operation, reducing database round trips.
- **BulkUpdate / BulkUpdateAsync:** This method updates many records in one batch, minimizing the number of database interactions and improving update performance.
- **BulkDelete / BulkDeleteAsync:** Deletes multiple records in a single database command, improving performance compared to individual deletes.
- **BulkInsertOrUpdate / BulkInsertOrUpdateAsync:** This performs a UPSERT operation, where records are inserted if they do not exist or updated if they do. It combines insert and update operations (UPSERT) for handling records with existing primary keys.
- **BulkSaveChanges / BulkSaveChangesAsync:** Batches multiple operations (INSERT, UPDATE, DELETE) into a single database round trip, enhancing performance significantly.

Install EFCore.BulkExtensions Package

First, you need to install the EFCore.BulkExtensions package in your project. You can install it via NuGet Package Manager or by executing the following command in the Package Manager Console:

Install-Package EFCore.BulkExtensions

Once installed, you can verify the package inside your solution's Packages folder, as shown in the image below.



Important Note: If your project includes the `Z.EntityFramework.Extensions.EFCore` package, you should remove it to avoid conflicts with `EFCore.BulkExtensions`. These packages may have overlapping functionalities, leading to unexpected behavior during bulk operations.

Setting Up the Entity and DbContext

To demonstrate bulk operations using `EFCore.BulkExtensions`, we will use a simple `Student` entity and configure the `DbContext` accordingly.

Student Entity

Create a class file named **Student.cs** in the Entities folder and add the following code:

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Student
    {
        public int StudentId { get; set; }
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public string Branch { get; set; }
    }
}
```

Configuring the DbContext

Modify your `EFCoreDbContext` class to include the `Students DbSet` property, which maps to the `Students` table in the database. Additionally, configure the database connection string. Update the `EFCoreDbContext` class as follows:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
```

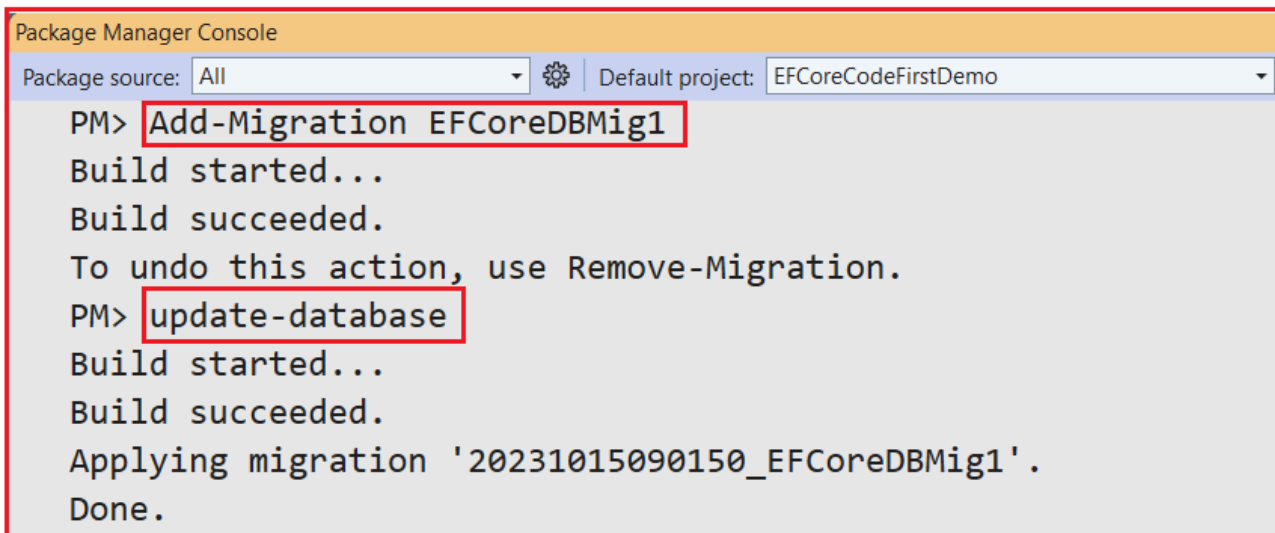
```

{
    // Configure the SQL Server connection string
    optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServ
}
// DbSet representing the Students table
public DbSet<Student> Students { get; set; }
}
}

```

Applying Migrations and Updating the Database

After setting up the Student entity and DbContext, the necessary migrations are applied to create the database schema. Open the Package Manager Console and execute the Add-Migration and Update-Database commands as follows:

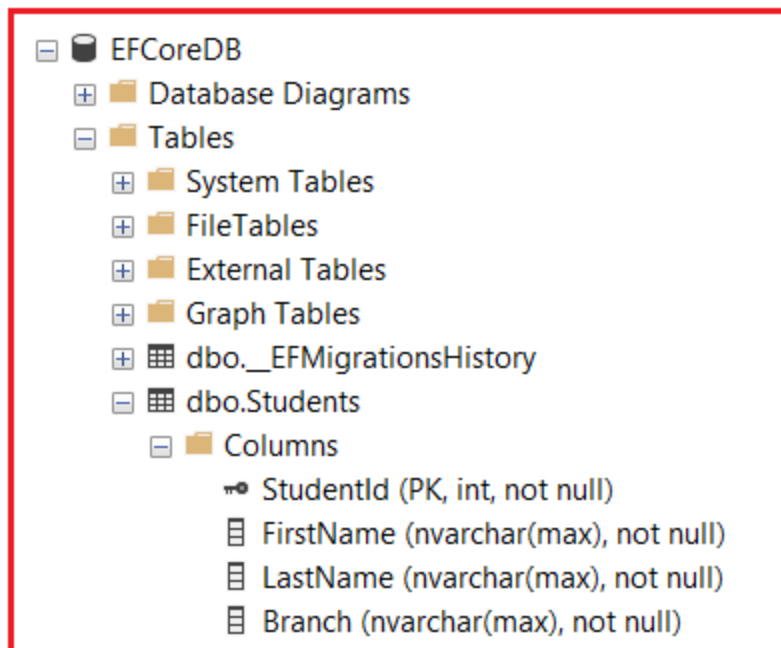


```

Package Manager Console
Package source: All [v] Default project: EFCoreCodeFirstDemo [v]
PM> Add-Migration EFCoreDBMig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Applying migration '20231015090150_EFCoreDBMig1'.
Done.

```

These commands will create the necessary tables in the specified database based on the Student entity, as shown in the below image:



Performing Bulk Insert Operations using EFCore.BulkExtensions

EFCore.BulkExtensions provides the BulkInsert and BulkInsertAsync methods to efficiently insert large numbers of entities into the database in a single operation. This approach minimizes database round trips and enhances performance.

For a better understanding, please modify the Program class as follows. In this example, we insert multiple students into the database using the BulkInsert extension method. Notice that we don't need to call the standard SaveChanges method when performing bulk insert operations.

```
using Microsoft.EntityFrameworkCore;
using EFCoreCodeFirstDemo.Entities;
using EFCore.BulkExtensions;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            try
            {
                // Create a list of new students to insert
                List<Student> newStudents = new List<Student>()
                {
                    new Student() { FirstName = "Pranaya", LastName = "Rout",
Branch = "CSE" },
                    new Student() { FirstName = "Hina", LastName = "Sharma",
Branch = "CSE" },
                    new Student() { FirstName = "Anurag", LastName =
"Mohanty", Branch = "CSE" },
                    new Student() { FirstName = "Prity", LastName = "Tiwary",
Branch = "ETC" }
                };
                using var context = new EFCoreDbContext();
                // Perform Bulk Insert using EFCore.BulkExtensions
                context.BulkInsert(newStudents);
                // No Need for SaveChanges():
                // The BulkInsert method handles database interactions
internally, eliminating the need to call SaveChanges().
                Console.WriteLine("BulkInsert: Successfully inserted new
students.");
                // Display all students belonging to the CSE branch
                DisplayStudentsByBranch("CSE");
                Console.ReadKey();
            }
            catch (Exception ex)
            {
                Console.WriteLine($"BulkInsert Error: {ex.Message}");
            }
        }
        // Retrieves and displays students from a specified branch.
    }
}
```

```

public static void DisplayStudentsByBranch(string branch)
{
    using var context = new EFCoreDbContext();
    // Fetch all students where Branch matches the specified value
    var studentsList = context.Students
        .AsNoTracking() // Improves performance
for read-only operations
        .Where(std => std.Branch == branch)
        .ToList();
    Console.WriteLine($"\\nStudents in {branch} Branch:");
    foreach (var student in studentsList)
    {
        Console.WriteLine($"\\tStudent ID: {student.StudentId}, Name:
{student.FirstName} {student.LastName}, Branch: {student.Branch}");
    }
}
}
}

```

Explanation:

- **Creating Student List:** A list of new Student objects is created to be inserted into the database.
- **BulkInsert Method:** The BulkInsert method inserts all records in the newStudents list into the database in one operation.
- **No SaveChanges Needed:** Unlike standard EF Core operations, BulkInsert manages its own transactions, so there's no need to call SaveChanges().
- **Display Method:** After insertion, the DisplayStudentsByBranch method retrieves and displays students from the specified branch to verify the operation.

Output:

```

BulkInsert: Successfully inserted new students.

Students in CSE Branch:
    Student ID: 1, Name: Pranaya Rout, Branch: CSE
    Student ID: 2, Name: Hina Sharma, Branch: CSE
    Student ID: 3, Name: Anurag Mohanty, Branch: CSE

```

Note: The SQL Profiler will show the bulk operation using optimized SQL INSERT statements.

Performing Bulk Update Operations using EFCore.BulkExtensions

EFCore.BulkExtensions provides the BulkUpdate and BulkUpdateAsync methods, allowing us to update a large number of entities in the database efficiently. For a better understanding, please modify the Program class as follows. In this example, we first fetch all students whose Branch is CSE, update their FirstName and LastName, and then apply the updates using the BulkUpdate extension method.

```

using EFCore.BulkExtensions;
using Microsoft.EntityFrameworkCore;
using EFCoreCodeFirstDemo.Entities;

```

```

namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            try
            {
                Console.WriteLine("Starting BulkUpdate Operation...");
                // Specify the branch to update
                string branchToUpdate = "CSE";
                // Perform Bulk Update
                BulkUpdateStudents(branchToUpdate);
                Console.WriteLine("BulkUpdate: Successfully updated student
records.");

                // Display updated students to verify changes
                DisplayStudentsByBranch(branchToUpdate);
                Console.ReadKey();
            }
            catch (Exception ex)
            {
                Console.WriteLine($"BulkUpdate Error: {ex.Message}");
            }
        }
        // Updates the first and last names of students in the specified
        branch.
        public static void BulkUpdateStudents(string branch)
        {
            using var context = new EFCoreDbContext();
            // Fetch students belonging to the specified branch
            var studentsToUpdate = context.Students
                .Where(std => std.Branch == branch)
                .ToList();
            // Modify the desired properties for each student
            foreach (var student in studentsToUpdate)
            {
                student.FirstName += " Updated";
                student.LastName += " Updated";
            }
            // Perform Bulk Update using EFCore.BulkExtensions
            context.BulkUpdate(studentsToUpdate);
        }
        // Retrieves and displays students from a specified branch.
        public static void DisplayStudentsByBranch(string branch)
        {
            using var context = new EFCoreDbContext();
            // Fetch all students where Branch matches the specified value
            var studentsList = context.Students
                .AsNoTracking() // Improves performance
                .Where(std => std.Branch == branch)
                for read-only operations
        }
    }
}

```

```

                .ToList();
        Console.WriteLine($"\\nUpdated Students in {branch} Branch:");
        foreach (var student in studentsList)
        {
            Console.WriteLine($"\\tID: {student.StudentId}, First Name:
{student.FirstName}, Last Name: {student.LastName}, Branch:
{student.Branch}");
        }
    }
}
}

```

Explanation:

- **Selecting Students:** Retrieves all students from the specified branch (CSE in this case).
- **Modifying Properties:** Appends " Updated" to both FirstName and LastName of each selected student.
- **BulkUpdate Method:** Applies all modifications to the database in a single, efficient operation.
- **Verification:** Displays the updated student records to confirm the changes.

Output:

```

Starting BulkUpdate Operation...
BulkUpdate: Successfully updated student records.

Updated Students in CSE Branch:
ID: 1, First Name: Pranaya Updated, Last Name: Rout Updated, Branch: CSE
ID: 2, First Name: Hina Updated, Last Name: Sharma Updated, Branch: CSE
ID: 3, First Name: Anurag Updated, Last Name: Mohanty Updated, Branch: CSE

```

Performing Bulk Delete using EFCore.BulkExtensions

EFCore.BulkExtensions provides the BulkDelete and BulkDeleteAsync methods, allowing you to delete a large number of entities efficiently in a single operation. For a better understanding, please modify the Program class as follows. In this example, we fetch all students where the Branch is ETC and delete them using the BulkDelete extension method.

```

using EFCore.BulkExtensions;
using Microsoft.EntityFrameworkCore;
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            try
            {
                Console.WriteLine("Starting BulkDelete Operation...");
                // Specify the branch to delete students from
                string branchToDelete = "ETC";
                // Perform Bulk Delete
                BulkDeleteStudents(branchToDelete);
            }
            catch { }
        }
    }
}

```

```

        Console.WriteLine("BulkDelete: Successfully deleted student
records.");
        // Display remaining students to verify deletion
        DisplayStudentsByBranch("CSE");
        DisplayStudentsByBranch("ETC");
        Console.ReadKey();
    }
    catch (Exception ex)
    {
        Console.WriteLine($"BulkDelete Error: {ex.Message}");
    }
}
// Deletes all students belonging to the specified branch.
public static void BulkDeleteStudents(string branch)
{
    using var context = new EFCoreDbContext();
    // Fetch students belonging to the specified branch
    var studentsToDelete = context.Students
        .Where(std => std.Branch == branch)
        .ToList();
    // Perform Bulk Delete using EFCore.BulkExtensions
    context.BulkDelete(studentsToDelete);
}
// Retrieves and displays students from a specified branch.
public static void DisplayStudentsByBranch(string branch)
{
    using var context = new EFCoreDbContext();
    // Fetch all students where Branch matches the specified value
    var studentsList = context.Students
        .AsNoTracking() // Improves performance
        for read-only operations
        .Where(std => std.Branch == branch)
        .ToList();

    if (studentsList.Any())
    {
        Console.WriteLine($"
Students in {branch} Branch:");
        foreach (var student in studentsList)
        {
            Console.WriteLine($"
ID: {student.StudentId}, Name:
{student.FirstName} {student.LastName}, Branch: {student.Branch}");
        }
    }
    else
    {
        Console.WriteLine($"
No students found in {branch}
Branch.");
    }
}
}
}
}

```


Explanation:

- **Selecting Students to Delete:** Retrieves all students from the specified branch (ETC).
- **BulkDelete Method:** Deletes all selected student records in a single operation.
- **Verification:** Displays remaining students in both CSE and ETC branches to confirm deletion.

Output:

```
Starting BulkDelete Operation...
BulkDelete: Successfully deleted student records.

Students in CSE Branch:
    ID: 1, Name: Pranaya Updated Rout Updated, Branch: CSE
    ID: 2, Name: Hina Updated Sharma Updated, Branch: CSE
    ID: 3, Name: Anurag Updated Mohanty Updated, Branch: CSE

No students found in ETC Branch.
```

Asynchronous Bulk Operations with EFCore.BulkExtensions

Implementing asynchronous bulk operations using EFCore.BulkExtensions ensures non-blocking database interactions, enhancing application performance and responsiveness. Asynchronous methods like BulkInsertAsync, BulkUpdateAsync, and BulkDeleteAsync allow operations to run without freezing the main thread, which is particularly beneficial in UI applications or services handling multiple concurrent requests.

Before proceeding, ensure that the Students table has relevant data. If necessary, you can clear the table using the following SQL statement:

```
TRUNCATE TABLE Students;
```

Example to Understand Asynchronous Bulk Operations using EFCore.BulkExtensions Package

Modify the Program class as follows to perform bulk operations asynchronously using EFCore.BulkExtensions. In this example, we use asynchronous methods like BulkInsertAsync, BulkUpdateAsync, and BulkDeleteAsync to ensure the main thread remains responsive during execution.

```
using EFCore.BulkExtensions;
using Microsoft.EntityFrameworkCore;
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreBulkDemo
{
    public class Program
    {
        static async Task Main(string[] args)
        {
            Console.WriteLine("=== EF Core Asynchronous CRUD Operations with EFCore.BulkExtensions ===\n");
            try
            {
```

```

        // Initialize the DbContext
        using var context = new EFCoreDbContext();
        // 1. Create (Bulk Insert) Operation
        await BulkInsertStudentsAsync(context);
        // 2. Read Operation
        await DisplayStudentsByBranchAsync(context, "CSE");
        // 3. Update (Bulk Update) Operation
        await BulkUpdateStudentsAsync(context, "CSE");
        // 4. Read Operation to verify updates
        await DisplayStudentsByBranchAsync(context, "CSE");
        // 5. Delete (Bulk Delete) Operation
        await BulkDeleteStudentsAsync(context, "ETC");
        // 6. Read Operations to verify deletions
        await DisplayStudentsByBranchAsync(context, "CSE");
        await DisplayStudentsByBranchAsync(context, "ETC");
    }
    catch (Exception ex)
    {
        // Handle any unexpected exceptions
        Console.WriteLine($"An unexpected error occurred:
{ex.Message}");
    }
}
// Performs bulk insert of new students asynchronously.
private static async Task BulkInsertStudentsAsync(EFCoreDbContext
context)
{
    Console.WriteLine("1. Starting Bulk Insert Operation...");
    // Define a list of new students to insert
    List<Student> newStudents = new List<Student>()
    {
        new Student() { FirstName = "Alice", LastName = "Johnson",
Branch = "CSE" },
        new Student() { FirstName = "Bob", LastName = "Smith", Branch
= "CSE" },
        new Student() { FirstName = "Charlie", LastName = "Brown",
Branch = "ETC" },
        new Student() { FirstName = "Diana", LastName = "Prince",
Branch = "ETC" }
    };
    try
    {
        // Perform Bulk Insert asynchronously
        await context.BulkInsertAsync(newStudents);
        Console.WriteLine("Bulk Insert: Successfully inserted new
students.\n");
    }
    catch (Exception ex)
    {
        // Handle exceptions related to bulk insert
        Console.WriteLine($"Bulk Insert Error: {ex.Message}\n");
    }
}

```

```

    }
}
// Displays students belonging to a specific branch asynchronously.
private static async Task
DisplayStudentsByBranchAsync(EFCoreDbContext context, string branch)
{
    Console.WriteLine($"2. Retrieving Students in '{branch}'
Branch...\n");
    try
    {
        // Fetch students asynchronously with no tracking for
performance
        var studentsList = await context.Students
                                .AsNoTracking()
                                .Where(std => std.Branch ==
branch)
                                .ToListAsync();

        if (studentsList.Any())
        {
            Console.WriteLine($"Students in '{branch}' Branch:");
            Console.WriteLine("-----");
            foreach (var student in studentsList)
            {
                Console.WriteLine($"tID: {student.StudentId}, Name:
{student.FirstName} {student.LastName}, Branch: {student.Branch}");
            }
            Console.WriteLine();
        }
        else
        {
            Console.WriteLine($"No students found in '{branch}'
Branch.\n");
        }
    }
    catch (Exception ex)
    {
        // Handle exceptions related to data retrieval
        Console.WriteLine($"Read Error: {ex.Message}\n");
    }
}
// Performs bulk update of students' names asynchronously.
private static async Task BulkUpdateStudentsAsync(EFCoreDbContext
context, string branch)
{
    Console.WriteLine("3. Starting Bulk Update Operation...");
    try
    {
        // Fetch students asynchronously
        var studentsToUpdate = await context.Students

```

```

        .Where(std => std.Branch
        == branch)
        .ToListAsync();

    if (studentsToUpdate.Any())
    {
        // Modify the desired properties for each student
        foreach (var student in studentsToUpdate)
        {
            student.FirstName += " Updated";
            student.LastName += " Updated";
        }
        // Perform Bulk Update asynchronously
        await context.BulkUpdateAsync(studentsToUpdate);
        Console.WriteLine("Bulk Update: Successfully updated
student records.\n");
    }
    else
    {
        Console.WriteLine($"No students found in '{branch}'
Branch to update.\n");
    }
}
catch (Exception ex)
{
    // Handle exceptions related to bulk update
    Console.WriteLine($"Bulk Update Error: {ex.Message}\n");
}
}
// Performs bulk delete of students belonging to a specific branch
asynchronously.
private static async Task BulkDeleteStudentsAsync(EFCoreDbContext
context, string branch)
{
    Console.WriteLine("4. Starting Bulk Delete Operation...");
    try
    {
        // Fetch students asynchronously
        var studentsToDelete = await context.Students
        .Where(std => std.Branch
        == branch)
        .ToListAsync();

        if (studentsToDelete.Any())
        {
            // Perform Bulk Delete asynchronously
            await context.BulkDeleteAsync(studentsToDelete);
            Console.WriteLine($"Bulk Delete: Successfully deleted
students from '{branch}' Branch.\n");
        }
        else
        {

```

```

        Console.WriteLine($"No students found in '{branch}'
Branch to delete.\n");
    }
}
catch (Exception ex)
{
    // Handle exceptions related to bulk delete
    Console.WriteLine($"Bulk Delete Error: {ex.Message}\n");
}
}
}
}
}

```

Explanation of the Code

- **Bulk Insert:** Inserts four students into the database asynchronously using BulkInsertAsync.
- **Read Operation:** Displays students in the CSE branch asynchronously using DisplayStudentsByBranchAsync.
- **Bulk Update:** This function appends " Updated" to the first and last names of students in the CSE branch asynchronously using BulkUpdateAsync.
- **Read Operation:** Verifies that the update was successful by displaying updated students.
- **Bulk Delete:** Deletes students in the ETC branch asynchronously using BulkDeleteAsync.
- **Read Operations:** Confirms the deletion by attempting to retrieve students from both CSE and ETC branches.

Output:

```

=== EF Core Asynchronous CRUD Operations with EFCore.BulkExtensions ===

1. Starting Bulk Insert Operation...
Bulk Insert: Successfully inserted new students.

2. Retrieving Students in 'CSE' Branch...

Students in 'CSE' Branch:
-----
ID: 1, Name: Alice Johnson, Branch: CSE
ID: 2, Name: Bob Smith, Branch: CSE

3. Starting Bulk Update Operation...
Bulk Update: Successfully updated student records.

2. Retrieving Students in 'CSE' Branch...

Students in 'CSE' Branch:
-----
ID: 1, Name: Alice Updated Johnson Updated, Branch: CSE
ID: 2, Name: Bob Updated Smith Updated, Branch: CSE

4. Starting Bulk Delete Operation...
Bulk Delete: Successfully deleted students from 'ETC' Branch.

2. Retrieving Students in 'CSE' Branch...

Students in 'CSE' Branch:
-----
ID: 1, Name: Alice Updated Johnson Updated, Branch: CSE
ID: 2, Name: Bob Updated Smith Updated, Branch: CSE

2. Retrieving Students in 'ETC' Branch...

No students found in 'ETC' Branch.

```

Bulk Insert or Update (UPSERT) using EFCore.BulkExtensions

EFCore.BulkExtensions provides the BulkInsertOrUpdate and BulkInsertOrUpdateAsync methods, allowing us to perform UPSERT operations. UPSERT operations combine insert and update functionalities based on the presence of primary keys. Existing records (matched by primary keys) are updated while new records are inserted.

For a better understanding, please modify the Program class as follows. In this example, we will bulk insert or update a list of students. Existing students (matched by StudentId) will be updated, while new students will be inserted.

```

using EFCore.BulkExtensions;
using Microsoft.EntityFrameworkCore;
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static async Task Main(string[] args)
        {
            try
            {
                Console.WriteLine("Starting BulkInsertOrUpdate
operation...");
            }
            catch { }
        }
    }
}

```

```

        // Perform Bulk InsertOrUpdate operation
        await BulkInsertOrUpdateAsync();
        Console.WriteLine("BulkInsertOrUpdate operation completed.");
    }
    catch (Exception ex)
    {
        Console.WriteLine($"An error occurred: {ex.Message}");
    }
}

private static async Task BulkInsertOrUpdateAsync()
{
    using var context = new EFCoreDbContext();
    // Existing list of students to merge (sync) with the database
    List<Student> studentsToMerge = new List<Student>
    {
        // Assume that StudentId 1 exists and will be updated
        new Student { StudentId = 1, FirstName = "John", LastName =
"Doe Updated", Branch = "CSE" },

        // This is a new student that will be inserted
        new Student { FirstName = "Ramesh", LastName = "Sethy",
Branch = "CSE" }
    };
    // Perform BulkInsertOrUpdate operation:
    // - Updates student with StudentId = 1
    // - Inserts new student without an ID
    await context.BulkInsertOrUpdateAsync(studentsToMerge);
    // Display current students in the database after the merge
    await DisplayStudentsAsync();
}

private static async Task DisplayStudentsAsync()
{
    using var context = new EFCoreDbContext();
    var students = await context.Students.ToListAsync();
    Console.WriteLine("Current Students in the database:");
    foreach (var student in students)
    {
        Console.WriteLine($"  \tID: {student.StudentId}, Name:
{student.FirstName} {student.LastName}, Branch: {student.Branch}");
    }
    Console.WriteLine(); // Blank line for better readability
}
}
}

```

Explanation:

- **Existing Student:** A Student object with StudentId = 1 is provided. EFCore.BulkExtensions identifies this as an existing record and updates it.
- **New Student:** A Student object without a StudentId (assuming it's auto-generated) is provided. EFCore.BulkExtensions insert this as a new record.

- **BulkInsertOrUpdateAsync Method:** Merges the provided list with the existing database records, performing updates or inserts as necessary.
- **Verification:** Displays all current students in the database after the upsert operation to confirm changes.

Output:

```
Starting BulkInsertOrUpdate operation...
Current Students in the database:
    ID: 1, Name: John Doe Updated, Branch: CSE
    ID: 2, Name: Bob Updated Smith Updated, Branch: CSE
    ID: 5, Name: Ramesh Sethy, Branch: CSE

BulkInsertOrUpdate operation completed.
```

Important Notes:

- **Primary Keys:** Ensure that the entities have correctly defined primary keys (StudentId in this case) for UPSERT operations to work as expected.
- **Auto-Increment IDs:** If StudentId is auto-incremented by the database, new entities should omit this field or set it to the default value.

BulkSaveChanges Example using Student Entity

The BulkSaveChanges method batches multiple INSERT, UPDATE, and DELETE operations when calling BulkSaveChanges, reducing database round trips and improving performance when handling large datasets.

For a better understanding, please modify the program class as follows. In this example, we update, insert, and delete multiple students and then call the BulkSaveChanges method to save all changes efficiently in one go.

```
using Microsoft.EntityFrameworkCore;
using EFCoreCodeFirstDemo.Entities;
using EFCore.BulkExtensions;
namespace EFCoreCodeFirstDemo
{
    internal class Program
    {
        static async Task Main(string[] args)
        {
            try
            {
                Console.WriteLine("Starting BulkSaveChanges operation...");
                // Perform Bulk SaveChanges operation
                await BulkSaveChangesAsync();
                Console.WriteLine("BulkSaveChanges operation completed.");
            }
            catch (Exception ex)
            {
                Console.WriteLine($"An error occurred: {ex.Message}");
            }
        }
    }
}
```



```

private static async Task BulkSaveChangesAsync()
{
    using var context = new EFCoreDbContext();
    // Fetch existing students from the database
    var existingStudents = await context.Students.Where(s => s.Branch
== "CSE").ToListAsync();
    // Updating existing students (append 'Updated' to their names)
    foreach (var student in existingStudents)
    {
        student.FirstName += " Updated";
        student.LastName += " Updated";
    }
    // Adding a new student to the context
    context.Students.Add(new Student { FirstName = "New", LastName =
"Student", Branch = "ETC" });
    // Deleting a student from the context (this will be batched in
the BulkSaveChanges)
    var studentToDelete = await context.Students.Where(s =>
s.FirstName == "Sethy").FirstOrDefaultAsync();
    if (studentToDelete != null)
    {
        context.Students.Remove(studentToDelete);
    }
    // Perform BulkSaveChanges to save all updates, inserts, and
deletes in one go
    await context.BulkSaveChangesAsync();
    // Display current students in the database after the save
    await DisplayStudentsAsync();
}

private static async Task DisplayStudentsAsync()
{
    using var context = new EFCoreDbContext();
    var students = await context.Students.ToListAsync();
    Console.WriteLine("Current Students in the database:");
    foreach (var student in students)
    {
        Console.WriteLine($"ID: {student.StudentId}, Name:
{student.FirstName} {student.LastName}, Branch: {student.Branch}");
    }
    Console.WriteLine(); // Blank line for better readability
}
}
}

```

Important Notes:

- **Transaction Management:** BulkSaveChangesAsync handles transactions internally, ensuring that all operations are executed successfully or rolled back in case of errors.
- **Performance:** This method is particularly beneficial when multiple types of operations need to be performed together, as it reduces the number of database round trips

In the next article, I will discuss [Performance Benchmarks Between Standard EF Core Bulk Operation vs. EFCore.BulkExtensions vs. Z.EntityFramework.Extensions.EFCore](#) with Examples. In this article, I explain **Bulk Operations in Entity Framework Core** using **EFCore.BulkExtensions** with examples. I hope you enjoyed this article.